

Resumen: APIs y Sockets

Fastbook 04 · Core Desarrollo — Autor original: Jaime Morales

Este documento resume los conceptos clave del fastbook sobre canales de comunicación entre aplicaciones: qué es una API, las arquitecturas REST y SOAP, los sockets, y una comparativa práctica entre REST y SOAP.

1. APIs: concepto y tipos

Una **API** (Application Programming Interface) es una pieza de código que permite que las aplicaciones se comuniquen entre sí y compartan información y funcionalidades. Lo esencial: **es un módulo de software que interactúa o se comunica con otro**.

Tipos de APIs según su acceso

- **Privadas:** pertenecen a una empresa (ej. un banco); solo personas autorizadas pueden acceder.
- **Públicas:** abiertas a cualquier ciudadano/desarrollador sin validación (ej. API de la Comunidad de Madrid para eventos o el tiempo).
- **De entorno local:** solo consumibles desde una red interna privada; suelen ampliar funcionalidad o proteger información confidencial.

API vs. Endpoint

Son conceptos distintos que suelen confundirse: **la API es la interfaz genérica** que ofrece una o varias funcionalidades, mientras que el **endpoint es la URL concreta** a la que se accede para ejecutar una de esas funcionalidades (request → response).

2. REST

Según Red Hat, una API REST es una interfaz de programación que se ajusta a los límites de la arquitectura REST, permitiendo la interacción con servicios web RESTful. Se establece un contrato entre el **consumidor** (quien llama) y el **productor** (quien expone la llamada).

Importante: REST no es un protocolo ni un estándar, sino un conjunto de límites (constraints) que definen la arquitectura de la aplicación.

Formatos de envío de datos

JSON (el más usado), HTML, XLT, Python, PHP, o sin formato.

Características para que una API sea REST

- **Arquitectura cliente-servidor:** clientes, servidores y recursos gestionados vía HTTP.

- **Sin estado (stateless):** no se guarda información del cliente entre solicitudes; cada petición es independiente.
- **Cacheable:** los datos pueden almacenarse en caché para optimizar interacciones.
- **Interfaz uniforme**, que exige 4 condiciones:
 - 1. Los recursos deben ser identificables e independientes de su representación.
 - 2. El cliente debe poder manipular recursos a través de la representación recibida.
 - 3. Mensajes autodescriptivos: deben indicar cómo procesarlos.
 - 4. Uso de hipertexto/hipermedios (HATEOAS): hipervínculos para descubrir acciones disponibles.
- **Sistema en capas:** jerarquías de servidores (seguridad, balanceo de carga) invisibles para el cliente.
- **Código bajo demanda (opcional):** el servidor puede enviar código ejecutable al cliente.

Herramientas de documentación de APIs REST

- **Swagger:** documenta reglas de la API (contrato, parámetros, tipos). Swagger UI la hace interactiva vía JSON/YAML.
- **RAML:** (Restful API Modeling Language) lenguaje sencillo para modelar aplicaciones REST.
- **OpenAPI:** estándar (nacido de Swagger) para describir APIs de forma uniforme y legible por máquina, usando YAML o JSON.

3. SOAP

SOAP (Simple Object Access Protocol) surgió a inicios de los años 90 con el objetivo de posibilitar la comunicación entre un cliente y el navegador de internet, funcionando sobre protocolos como HTTP, HTTPS, FTP o TCP.

Clave: el framework SOAP se basa en el metalenguaje XML, por lo que toda solicitud es un documento XML bien formado, envuelto en un *envelope* (sobre).

Estructura del mensaje SOAP

Elemento	Descripción
Header	Encabezado con metadatos (ej. tipo de cifrado). Su uso es opcional.
Body	Cuerpo del mensaje con los datos reales; depende de la aplicación, no de SOAP.

4. Comparativa: REST vs. SOAP

Criterio	SOAP	REST
Significado	Protocolo simple de acceso a objetos	Transferencia de estado representacional
¿Qué es?	Protocolo para comunicación entre aplicaciones	Estilo de arquitectura para diseñar interfaces
Diseño	Expone la operación	Expone los datos
Protocolo transporte	Cualquiera (HTTP, FTP, TCP...)	Solo HTTP/HTTPS
Formato de datos	Solo XML	XML, JSON, texto plano, HTML
Rendimiento	Más lento (mensajes grandes)	Más rápido (mensajes pequeños + caché)
Escalabilidad	Difícil (mantiene estado)	Fácil (sin estado)
Seguridad	Cifrado con sobrecarga adicional	Cifrado sin afectar rendimiento
Caso de uso	Apps antiguas y APIs privadas	Apps modernas y APIs públicas

5. Sockets

Nacieron en los años 80 para responder a la necesidad del sistema Unix de Berkeley de establecer comunicación entre procesos en red.

Un socket es el punto final de conexión: un elemento generado por el sistema operativo que permite el envío de información entre los procesos que lo utilizan. Su función principal es la **interconectividad**, permitiendo enviar y recibir datos entre procesos, incluso de diferentes máquinas.

Ventajas

- Intercambio de datos fiable y ordenado entre cliente y servidor.
- Se beneficia de la funcionalidad de las APIs.

Desventajas

- Depende de la respuesta del servidor.
- En sockets no orientados a conexión (UDP), no se garantiza el orden de llegada de los datos.

Principales tipos de sockets (según IBM)

Tipo	Descripción
SOCK_DGRAM	Mensajes cortos sin conexión, de longitud máxima fija. No garantiza orden ni fiabilidad; puede haber pérdida de datos.
SOCK_STREAM (secuencia de sockets)	Secuencias de bytes bidireccionales, fiables, en orden. En Internet se implementa sobre TCP/IP.
SOCK_SEQPACKET	Flujo de información secuenciado, fiable y no duplicado.

SOCK_CONN_DGRAM	Datagramas orientados a conexión: secuenciados y sin duplicados, pero no garantiza fiabilidad total
-----------------	---

6. Conclusiones clave

- Las **APIs REST** son más actuales, ágiles y consumen menos recursos: son el futuro predominante del desarrollo de interfaces web.
- **SOAP** sigue siendo relevante hoy en día, usado en muchos sectores para aplicaciones y servicios (especialmente en sistemas antiguos o entornos que exigen contratos estrictos).
- Los **sockets** son canales de intercomunicación entre procesos no relacionados, permitiendo el intercambio de datos tanto a nivel local como entre redes.